Problem A. 种树

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

小L喜欢种树。

小L在院子的左右两边各种了 d 棵树，左边第 i 棵树在第一天的高度为 x_i 米，每经过一天会长高 f_i 米，右边第 i 棵树在第一天的高度为 y_i ，每经过一天会长 g_i 米。

但是小L不喜欢树长得太高。为了保持美观，对于编号为 i 的树，如果树的高度达到或超过 m_i ，他会拿起剪刀剪掉 m_i 的高度，直到树的高度小于 m_i。

小L现在想知道，对于编号为 i 的树，在种下的前 n_i 天内，每天进行修剪过后，左边比右边低的时间有多少天。

注：第一天也会进行修剪。

Input

每个测试文件仅有一组测试数据。

第一行输入一个正整数 d $(1 \le d \le 10^4)$ ，表示道路两侧树的数量。

接下来 d 行，每行输入六个正整数 $f_i, x_i, g_i, y_i, n_i, m_i$ $(1 \le f_i, x_i, g_i, y_i, n_i, m_i \le 10^6)$ ，含义如题面所述。

Output

输出 d 行，每行一个非负整数，表示编号为 i 的树在 n_i 天内左边比右边低的天数。

Example

standard input	standard output
1 7 4 6 3 3 4	2

Note

对于第一组样例，一共要统计前 3 天：

第一天，道路左边的树初始高为 4 米，右边的树初始高为 3 米，左边的树超过了 4 米，所以要进行修剪，修剪后，左边的树为 0 米，右边的树为 3 米，左边低于右边；

第二天，道路左边的树高为 7 米，右边的树高为 9 米，他们都超过了 4 米，所以要进行修剪，修剪后，左边的树为 3 米，右边的树为 1 米，左边不低于右边；

第三天，道路左边的树高为 10 米，右边的树高为 7 米，他们都超过了 4 米，所以要进行修剪，修剪后，左边的树为 2 米，右边的树为 3 米，左边低于右边；

故输出 2 。

Problem B. 77G网络

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	1024 megabytes

喵国的地图可以视作有 n 个节点，$n-1$ 条边的有根树，节点代表城市，一条边代表连接两个城市的道路。首都位于根节点 1 号节点。

现在喵国要在一些城市安装全新的77G网络基站，为了保证每一条道路都有足够好的网络质量，对于每一条道路，它的两个端点城市中至少有一个要安装基站。然而，由于未知的技术问题，网络系统存在 m 个故障。故障 i 可以表述为：如果在城市 x_i 安装基站，那么在 x_i 的子树中，所有与 x_i 的距离恰好为 y_i 的城市都不能安装基站，否则基站间会产生互相干扰导致无法使用。

喵国请你帮忙选择一些城市，使得在这些城市安装基站可以满足上述所有要求。如果不存在合法方案，输出“No”。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 10^3)$ 。每组测试数据的格式如下。

第一行包含一个整数 n $(2 \le n \le 2 \times 10^5)$，表示城市数。

第二行包含 $n-1$ 个整数 $f_2, f_3, \ldots, f_n$ $(1 \le f_i < i)$ ，其中 f_i 表示第 i 个城市的父节点。

第三行包含一个整数 m $(0 \le m \le 2 \times 10^5)$，表示故障数。

接下来 m 行，每行包含两个整数 x_i 和 y_i $(1 \le x_i, y_i \le n)$，表示第 i 个故障。可能存在一模一样的两个故障。

在每个测试文件内，保证所有测试数据的 n 之和不超过 2×10^5 ，所有测试数据的 m 之和不超过 2×10^5。

Output

对于每组测试数据，如果没有合法方案，则输出一行 “No”。

否则，请输出三行，表示你的方案：

- 第一行输出 “Yes” 。
- 第二行输出一个整数 c ，表示你选择的城市数量。
- 第三行输出 c 个互不相同的整数，表示你选择的城市。

如果有多种合法方案，你可以输出任意一个。在输出方案时，你可以以任意顺序输出 c 个城市。

Example

standard input	standard output
3 6 1 1 2 2 4 2 1 3 2 1 6 1 1 2 2 4 6 1 1 1 2 1 3 2 1 2 2 2 1 8 1 1 3 1 1 2 4 10 1 2 3 2 1 3 5 1 6 1 3 2 4 1 3 2 5 1 3 1	Yes 4 5 3 4 1 No No

Problem C. 不对称填数

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

给定一个 $n \times m$ 的矩阵，并保证 $n \times m$ 能被 3 整除。你需要用 0、1 和 2 填满矩阵，每个数字恰好出现 $\frac{nm}{3}$ 次，使得：

- 水平方向或垂直方向不能出现连续 3 个相同的数字。
- 所有填 0 的单元格组成一个 8-连通区域 †，所有填 1 的单元格组成一个 8-连通区域，所有填 2 的单元格组成一个 8-连通区域。

† 如果两个单元格共享边或角，则称它们 8-邻接。一个单元格集合称为 8-连通，如果对于集合中任意两个单元格，可以找到一条路径连接它们，该路径完全在集合内，并且每一步都只从一个单元格移动到一个 8-邻接单元格。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 10^3)$ 。每组测试数据的格式如下。

第一行包含两个整数 n 和 m $(1 \le n, m \le 10^3, n \times m$ 能被 3 整除)，表示矩阵的行数和列数。

在每个测试文件内，保证所有测试数据的 $n \times m$ 之和不超过 10^6。

Output

对于每组测试数据，如果无法构造满足条件的矩阵，则输出一行 "No"。

否则，第一行输出 "Yes"，接下来输出矩阵的 n 行，每行 m 个数字。若存在多种解，你可以输出任意一种。

Example

standard input	standard output
2 3 4 1 12	Yes 0012 0021 2211 No

Problem D. 出奇制胜

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

给定一个无向二分图 $G = (U, V, E)$，其中 U 和 V 是两部分顶点集合，$E \subseteq U \times V$ 是边的集合。有若干顶点被标记为“禁用”，其他顶点为“启用”。图中有一颗棋子，先手玩家和后手玩家依次移动该棋子，先手从二分图左部 U 的 1 号点出发，移动规则如下：

- 在玩家的回合中，若棋子所在顶点为 x，须选择一条边 (x, y)，且顶点 y 必须处于启用状态；
- 玩家将棋子移动到 y 并将 x 标记为禁用；
- 若玩家无法找到可移动的边，则该玩家失败，对方获胜。

赛前可以对部分顶点状态执行“翻转”操作：将启用顶点改为禁用，或将禁用顶点改为启用。每个顶点最多翻转一次，起点不可被禁用。

请问，在双方均使用最优策略的情况下，为了使先手必胜，最少需要执行多少次翻转？如果无论如何也不可能使先手必胜，请输出 “-1” 。

Input

每个测试文件仅有一组测试数据。

第一行包含三个整数 n_U, n_V 和 m $(1 \le n_U, n_V \le 100,\ 0 \le m \le n_U \cdot n_V)$ ，分别表示左部 U 的大小、右部 V 的大小和连接两部的边集的大小。

接下来 m 行，每行两个整数 u 和 v $(1 \le u \le n_U,\ 1 \le v \le n_V)$ ，表示一条连接左部 u 号点和右部 v 号点的边。

接下来一行包含 n_U 个整数 $s_{u,1}, s_{u,2}, \ldots, s_{u,n_U}$ $(s_{u,i} \in \{0, 1\},\ s_{u,1} = 1)$ 。若 $s_{u,i} = 0$ ，则表示左部的 i 号点被禁用，否则表示该点被启用。

接下来一行包含 n_V 个整数 $s_{v,1}, s_{v,2}, \ldots, s_{v,n_V}$ $(s_{v,i} \in \{0, 1\})$ 。若 $s_{v,i} = 0$ ，则表示右部的 i 号点被禁用，否则表示该点被启用。

Output

输出一行一个整数，表示为了使先手必胜最少需要执行多少次翻转数量。如果无论如何也无法使先手必胜，请输出 “-1” 。

Examples

standard input	standard output
4 4 8 1 1 2 1 2 2 3 1 3 3 4 2 4 3 4 4 1 1 1 1 1 1 1 0	1
4 2 5 1 1 2 1 2 2 3 2 4 2 1 1 1 1 1 1	1
1 1 0 1 0	-1

Problem E. 变魔术

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

魔术师Link有一个 n 张牌构成的牌堆 s ，每张牌上有一个整数，表示这张牌的点数。

Link可以从牌堆中抽出两张牌。设牌上的点数为 x 和 y 。如果 $|x-y|=1$ ，则他可以变以下两种魔术之一：

- 将两张牌的点数变为 $x-1$ 和 $y-1$ ，然后放回牌堆；
- 将两张牌的点数变为 $x+1$ 和 $y+1$ ，然后放回牌堆。

你同样有一个包含 n 张牌的牌堆 t 。你想知道，Link能否通过任意次（可以是 0 次）魔术将他的牌变得和你手上的牌一样。请注意，牌堆中可出现点数一样的牌，且点数一样的牌之间没有任何区别。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 10^4)$ 。每组测试数据的格式如下。

第一行包含一个整数 n $(1 \le n \le 10^5)$ ，表示牌堆中牌的数量。

第二行包含 n 个整数 $s_1, s_2, \cdots, s_n$ $(1 \le s_i \le 10^9)$，表示 Link 的牌堆里的牌。

第三行包含 n 个整数 $t_1, t_2, \cdots, t_n$ $(1 \le t_i \le 10^9)$，表示你的牌堆里的牌。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^5 。

Output

对于每组数据，如果 Link 可以通过任意次（可以是 0 次）魔术将他的牌变得和你手上的牌一样，请输出 “Yes”；否则，请输出 “No” 。

Example

standard input	standard output
3 3 1 10 100 100 10 1 2 1 2 99 100 3 1 2 3 1 1 6	Yes Yes No

Note

在样例的第一组测试数据中，两个牌堆在初始状态下就完全相同，因此输出 “Yes” 。

在样例的第二组测试数据中，对两张牌连续执行 98 次魔术二，即可得到目标牌堆，因此输出 “Yes” 。

在样例的第三组测试数据中，目标牌堆是不可能变出的，因此输出 “No” 。

Problem F. 分治

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	1024 megabytes

Fall有一个长度为 n 的数组和 q 个针对该数组的询问，每个询问是一个区间 $[l_i, r_i]$ 。

Fall决定使用分治的方法处理这些询问。初始状态下，所有待处理的询问位于区间 $[1, n]$ 内，记为第 1 层。假设当前待处理的询问的区间在 $[L, R]$ 内，层数为 d ，他可以从 $[L, R]$ 中选择一个位置 x ，并在第 d 层处理所有询问区间包含 x 的询问。剩余的询问位于区间 $[L, x-1]$ 和 区间 $[x+1, R]$ 中，可以分别递归成两个子问题在第 $d+1$ 层处理（如果这两个区间内还有询问未处理的话）。

请你最小化询问被处理时的层数的最大值。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 10^4)$ 。每组测试数据的格式如下。

第一行包含两个整数 n 和 q $(1 \le n, q \le 10^6)$，表示数组的长度和询问的个数。

接下来 q 行，每行两个整数 l_i 和 r_i $(1 \le l_i \le r_i \le n)$ ，表示一个询问。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^6 ，所有测试数据的 q 之和不超过 10^6 。

Output

对于每组数据，输出一行一个整数，表示询问被处理时的层数的最大值的最小值。

Example

standard input	standard output
3 5 5 1 5 1 1 2 2 4 4 5 5 5 2 1 3 3 5 3 3 1 1 2 2 3 3	3 1 2

Note

对于样例的第一组数据，一种最优的分治方法为：选择 $x = 3$ ，在第 1 层处理了第一个询问，将第二、三个询问递归到 $[1, 2]$ 区间处理，将第四、五个询问递归到 $[4, 5]$ 区间处理。两个子区间内均还需要递归一次，因此询问被处理时的最大层数为 3 。

Problem G. 投影

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	1024 megabytes

给定一个 n 维的超球体和超立方体，定义如下：

超球体：$B \triangleq \{\mathbf{x} \mid \sum_{i=1}^{n} x_i^2 \le R^2; \mathbf{x} \in \mathbb{R}^n\}$；

超立方体：$C \triangleq \{\mathbf{x} \mid |x_i| \le l_i, \forall i \in [1,n]; \mathbf{x} \in \mathbb{R}^n\}$。

记 $S \triangleq B \cap C$。给定一个点 $\mathbf{x} = (x_1, x_2, \ldots, x_n)$，请找出集合 S 中（包含边界）离 $\mathbf{x}$ 最近的点，记为 $\mathbf{y} = (y_1, y_2, \ldots, y_n)$ 。

称点 $\mathbf{p}$ 比点 $\mathbf{q}$ 离 $\mathbf{x}$ 更近，当且仅当 $\sum_{i=1}^{n}(x_i - p_i)^2 \le \sum_{i=1}^{n}(x_i - q_i)^2$ 。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 3 \times 10^4)$ 。每组测试数据的格式如下。

第一行包含两个整数 n 和 a $(2 \le n \le 10^5, 1 \le a \le 10^9)$ ，其中 a 表示 R^2 的值。

第二行包含 n 个整数 $x_1, x_2, \ldots, x_n$ $(|x_i| \le 10^9)$。

第三行包含 n 个整数 $l_1, l_2, \ldots, l_n$ $(1 \le l_i \le 100)$。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^5 。

Output

对于每组测试数据，输出一行包含 n 个实数 $y_1, y_2, \ldots, y_n$。

当你的输出与标准答案的绝对误差或相对误差不超过 10^{-6} 时，你的输出将会被判定为正确。

具体地说，令你的答案为 y_i，标准答案为 y_i^*。你的答案被认为是正确的当且仅当 $\frac{|y_i - y_i^*|}{\max(1, |y_i^*|)} \le 10^{-6}$ 对所有 $i \in [1,n]$ 成立。

Example

standard input	standard output
3 2 25 1 10 4 4 2 25 10 1 4 4 3 6 3 3 3 2 2 2	1.0000000000 4.0000000000 4.0000000000 1.0000000000 1.4142135624 1.4142135624 1.4142135624

Problem H. 还原数组

Input file:	`standard input`
Output file:	`standard output`
Time limit:	4 seconds
Memory limit:	1024 megabytes

这是一道交互题。

小L有一个隐藏的大小为 n 的集合 $\{a_1, a_2, \ldots, a_n\}$ $(0 \le a_i < 2^{30})$，集合中元素的值两两不同。

为了猜出集合中元素的值，你可以向小L提出询问。在一个询问中，你可以给出一个非负整数 c ，小L会告诉你 $\max\{a_i \oplus c\}$ ，其中 $\oplus$ 表示按位异或 † 。

例如，$a = \{1, 4, 6\}$，$c = 3$，小L会告诉你 7 ，因为 $\max\{1 \oplus 3, 4 \oplus 3, 6 \oplus 3\} = \max\{2, 7, 5\} = 7$ 。

请使用不超过 $2n - 2$ 次询问猜出集合中的所有元素的值。

† 按位异或是指将两个二进制数字的每一位进行模 2 意义下的加法运算。例如： $(0011)_2 \oplus (0101)_2 = (0110)_2$ 。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 10^4)$ 。每组测试数据的格式如下。

每一组数据输入一个正整数 n $(2 \le n \le 10^5)$ ，表示集合的大小。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^5 。

Interaction Protocol

要提出询问，请以 "`?` c" $(0 \le c < 2^{30})$ 的格式输出你的询问。在清空输出缓冲区后，你的程序需要读入一行一个整数，表示 $\max\{a_i \oplus c\}$ 的值。在每组测试数据中，你只能提出不超过 $2n - 2$ 个询问。如果你提出了超出该数量的询问，评测结果将是未知的。

要猜想集合，请以 "`!` a_1 a_2 $\ldots$ a_n" $(0 \le a_i < 2^{30})$ 的格式输出你的猜测，集合中元素的顺序可以是任意的。在清空输出缓冲区之后，本组测试数据的交互结束。你需要立即开始处理下一组测试数据，或退出程序（如果已经处理完所有测试数据）。

请注意，本题的交互器是自适应的。这意味着集合中的元素并非在一开始就已经确定，而是会根据你的询问动态调整。交互器保证调整后的集合元素的值不会改变先前询问的回答。

清空输出缓冲区可以使用以下方式：

- C 和 C++ 使用 `fflush(stdout)` （如果您使用 `printf`）或 `cout.flush()` （如果您使用 `cout`）。
- Java 和 Kotlin 使用 `System.out.flush()` 。
- Python 使用 `stdout.flush()` 。

Example

standard input	standard output
1 4 7 7 7 7	 ? 0 ? 1 ? 5 ? 6 ! 1 2 6 7

Problem I. 套娃2

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

Alice 刚刚获得了 n 个大小互不相同的俄罗斯套娃。他将这些套娃从左到右摆成了一排，第 i 个位置上包含一个大小为 a_i 的套娃。

设第 i 格上的最小套娃的大小为 l_i ，最大套娃的大小为 r_i 。两个相邻格子 i 和 $i+1$ 上的套娃能合并，当且仅当 $r_i < l_{i+1}$ 或 $r_{i+1} < l_i$ 。新的套娃将包含原有第 i 格和第 $i+1$ 格上的所有套娃，并被放置在第 i 格上。所有编号大于 $i+1$ 的格子上的套娃都将向左移动一格以填补空缺。

例如，当 $n=5, a=[1,2,3,5,4]$ 时， Alice 可以:

1. 合并第 1 格和第 2 格上的套娃。此时剩余四个格子上的套娃大小分别为 $[(1,2),(3),(5),(4)]$ 。
2. 合并第 2 格和第 3 格上的套娃。此时剩余三个格子上的套娃大小分别为 $[(1,2),(3,5),(4)]$ 。
3. 合并第 1 格和第 2 格上的套娃。此时剩余两个格子上的套娃大小分别为 $[(1,2,3,5),(4)]$ 。根据规则，剩余的两个格子上的套娃无法再被合并。

Alice将持续进行合并操作，直到不能再进行任何合并操作为止。Alice至少要进行几次合并操作?

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 10^4)$ 。每组测试数据的格式如下。

第一行包含一个整数 n $(2 \le n \le 10^6)$，表示套娃的数量。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$ $(1 \le a_i \le n, \forall i \ne j, a_i \ne a_j)$ ，表示初始状态下每个位置上的套娃的大小。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^6 。

Output

对于每组数据，输出一行一个整数，表示最少要执行的合并操作数。

Example

standard input	standard output
5 5 1 2 3 5 4 5 1 2 3 4 5 5 1 3 5 4 2 5 1 5 2 4 3 5 1 2 4 3 5	3 4 3 2 3

Problem J. 环形匹配

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

给定两个字符串 s 和 t，字符串均由小写字母组成。

记 $s_{l,r}$ 表示 s 的第 l 个字符到第 r 个字符构成的子串，长度 $L = r - l + 1$ 。称 t 在 $s_{l,r}$ 的环上出现，当且仅当存在某个起点 p（$0 \le p < L$），使得对所有 $0 \le k < |t|$ 都有 $s_{l,r}[(p + k) \bmod L] = t[k]$ 。若 $L < |t|$，则认为 t 没有在 $s_{l,r}$ 的环上出现。

请统计有多少对不同的 (l, r) ，使得 t 在 $s_{l,r}$ 的环上出现。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 2 \times 10^5)$ 。每组测试数据的格式如下。

第一行一个字符串 s $(1 \le |s| \le 2 \times 10^5)$ 。

第二行一个字符串 t $(1 \le |t| \le 2 \times 10^5)$ 。

保证所有字符串仅包含小写字母。

在每个测试文件内，保证所有测试数据的 $|s|$ 之和不超过 2×10^5 ，所有测试数据的 $|t|$ 之和不超过 2×10^5 。

Output

对于每一组数据，输出一行一个整数，表示符合条件的 (l, r) 对数量。

Examples

standard input	standard output
3 cabcccabc ccabcc bcdabc abcd aaaxaaaxaaa axaaaxaa	3 3 10
4 aaaaaaaaa a abacab caba baaabaab aabaaab bbabbbabb bbbabbb	45 4 2 3

Problem K. 整理书架

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	1024 megabytes

给定两个长度为 n 的序列 a 和 b，你可以对这两个序列进行以下操作不超过 $3n$ 次：

选择任意的整数 i 和 j $(1 \le i, j \le n)$，然后交换 a_i 和 b_j。本次操作的代价为 $|i - j|$。

请寻找一个操作序列，使 a 和 b 相等，并最小化总代价。如果无法通过上述操作使两个序列相等，请输出 "-1"。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T $(1 \le T \le 2 \times 10^5)$ 。每组测试数据的格式如下。

第一行包含一个整数 n $(1 \le n \le 2 \times 10^5)$，表示序列的长度。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$ $(1 \le a_i \le n)$，表示序列 a。

第三行包含 n 个整数 $b_1, b_2, \ldots, b_n$ $(1 \le b_i \le n)$，表示序列 b。

在每个测试文件内，保证所有测试数据的 n 之和不超过 2×10^5 。

Output

对于每组数据，第一行输出两个整数 $cost$ 和 num $(0 \le num \le 3n)$，分别代表最小花费与对应的操作次数。如果无法通过上述操作使两个序列相等，请输出 "-1" 。

接下来 num 行，每行输出两个整数 x_i 和 y_i $(1 \le x_i, y_i \le n)$，表示第 i 步交换 a_{x_i} 和 b_{y_i} 。

可以证明，如果可以通过上述操作使两个序列相等，则一定存在一种操作次数不超过 $3n$ 且花费最小的方案。

Example

standard input	standard output
3 3 1 1 3 3 2 2 4 1 2 2 3 4 4 3 1 2 1 1 2 1	2 2 2 1 2 3 3 4 1 2 3 2 3 3 4 3 -1

Problem L. ICPC

Input file:	standard input
Output file:	standard output
Time limit:	8 seconds
Memory limit:	1024 megabytes

Fall正在参加国际复制粘贴挑战赛(International Copy & Paste Challenge, ICPC)。突然，后排的另一位同学站起身，将他的答题纸撕成了两半。Fall非常伤心，因为他得浪费时间与墨水将他的答案重新抄写一遍。除非... 他的所有答案都恰好在其中一半纸上?

来不及悲伤，Fall立刻在脑内开始算起了所有答案都恰好在其中一半纸上的概率。形式化地，答题纸可以被划分为 $n \times m$ 个网格状的单位正方形。纸上有 k 段Fall的笔迹。第 i 段笔迹可以表示为 (a_i, b_i)、(a_i, d_i)、(c_i, d_i)、(c_i, b_i) 围成的矩形，其中 $0 \le a_i < c_i \le n, 0 \le b_i < d_i \le m$ 。

Fall后排的同学将Fall的答题纸沿着一条"单调路径"撕开，并且这条路径是从所有单调路径中等概率随机选择的。所谓单调路径，指的是一条从 $(0, 0)$ 走到 (n, m) 的格点路径，每一步只能向右（+1 个单位的 x 方向）或向上（+1 个单位的 y 方向）。方便起见，即便这样的路径没能将答题纸撕成两半（即其中一半为空），也认为该路径属于所有单调路径之一。

说到这，Fall已经计算出了所有笔迹都位于其中一半纸上的概率了。不过他想验算一下，所以请你也计算出这个概率。为了避免Fall的后排同学看到这个概率后又再次撕毁Fall的答题纸，请你将答案对 $998\,244\,353$ 取模 † 后告诉Fall。

† 形式化地，设 $M = 998\,244\,353$。可以证明，答案可以表示为最简分数 $\frac{p}{q}$，其中 p 和 q 为整数，且 $q \not\equiv 0 \pmod{M}$。输出等于 $p \cdot q^{-1} \bmod M$ 的整数。换句话说，输出一个整数 x，使得 $0 \le x < M$ 且 $x \cdot q \equiv p \pmod{M}$。

Input

每个测试文件仅有一组测试数据。

第一行包含三个整数 n, m, k $(1 \le n, m, k \le 2 \times 10^5)$ ，表示纸张的大小和关键区域的数量。

接下来 k 行，每行四个整数 a_i, b_i, c_i, d_i $(0 \le a_i < c_i \le n,\ 0 \le b_i < d_i \le m)$ ，描述了一个关键区域。

Output

输出一行一个整数，表示所有笔迹都位于其中一半纸上的概率对 $998\,244\,353$ 取模的结果。

Examples

standard input	standard output
4 3 3 0 0 1 1 1 1 3 2 3 2 4 3	199648871
200000 200000 2 114 514 1919 810 233 666 77777 88888	243245654

Note

对于第一组样例，答案的分数形式为 $\frac{14}{35}$ 。

Problem M. 构造王国

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

给定一个长度为 n 的序列 $a_1, a_2, \ldots, a_n$ ，你需要构造一个 $n \times n$ 的矩阵 A ，使得对于任意两个集合 $S \subseteq S' \subseteq [n]$ ，

$$\det(A_{S'}) - \det(A_S) \equiv \left(\sum_{i \in S'} a_i - \sum_{i \in S} a_i \right) \pmod{998\,244\,353}.$$

其中，$[n]$ 表示集合 $\{1, 2, \ldots, n\}$ ，$\det(A)$ 表示 A 的行列式。对于一个子集 $T \subseteq [n]$，A_T 表示由索引属于 T 的行和列组成的 A 的主子矩阵。按照惯例，$\det(A_\emptyset) = 1$。

Input

每个测试文件仅有一组测试数据。

第一行包含一个整数 n $(1 \le n \le 18)$ ，表示序列的长度。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$ $(0 \le a_i < 998244353)$ ，表示序列中的元素。

Output

输出 n 行，第 i 行 n 个整数 $A_{i,1}, A_{i,2}, \ldots, A_{i,n}$ $(0 \le A_{i,j} < 998244353)$，表示你构造的矩阵 A 。

Example

standard input	standard output
2 6 8	7 12 4 9